Scope of Work — Domu Ops Copilot (POC → Production)

Author: Giovanna Souza Gomes **Status:** Proof-of-concept validated.

1. Purpose

The attached MVP proves a simple thing: that the repetitive parts of a Technical Operations Lead's week — turning a script into an agent, summarizing outcomes across clients, triaging flagged calls, fixing a poorly-behaving agent prompt, drafting engineering tickets, investigating a compliance concern, and checking calling-time compliance — can be meaningfully accelerated with an LLM in the loop (or, where an LLM isn't the right tool, with a simple rules engine), without needing a human to do everything from scratch each time.

It intentionally does **not** prove that this is safe or reliable to run unsupervised, at Domu's actual scale (millions of calls/week, regulated financial clients), on real data. That gap is what this document scopes.

2. What the MVP proves vs. doesn't

Proven	Not proven
The LLM can convert a raw script (or a transcribed call recording) into a usable, branching call flow + agent prompt	Whether that prompt performs well in a *live* voice call (latency, interruption handling, tone)

The LLM can suggest a QA category, and a human can confirm/override it in one click	Accuracy against a labeled dataset of real flagged calls, at scale
The LLM can diagnose a prompt problem and propose a fix without rewriting the rest of the prompt	Whether the fix actually improves the agent's real-call behavior — needs a regression suite, not just a plausible-looking diff
A request → ticket generator produces readable, structured output	Whether developers actually find AI-drafted tickets accurate enough to act on without rewriting
The LLM can produce a reasonable first-pass compliance risk summary	Whether that summary is *correct* — this must never be trusted without a human legal/ compliance reviewer
A rules engine can flag calling-hour/holiday violations deterministically	Whether the specific hour windows and holiday list used are legally accurate for each region — they're illustrative demo data here
A wrapper with timeout + retry + fallback keeps the UI from breaking when the LLM fails	Behavior under real production load, concurrent users, or sustained API outages

3. Production scope, per feature

3.1 Script → structured agent (Task 1)

The MVP already treats the generated call flow and prompt as an editable draft, not a final artifact — every step, branch, and the prompt text itself can be changed, steps/branches can be added or removed, and the result exports to Markdown, plain text, or PDF (whichever's

easiest to drop into an email, a ticket, or a doc). That editability is the right instinct; production needs to carry it forward into a real review workflow rather than "edit locally, download a file":

- **Data**: Replace the free-text-in / free-text-out flow with a proper agent-config schema that matches whatever the real voice runtime consumes (LiveKit/Twilio agent config, or Domu's internal format) — not just a prompt string. This includes the branching structure the MVP already generates (conditional steps like objection handling) — production needs that represented as real branch/state logic the voice runtime can execute, not just descriptive text for a human to read.
- **Human review gate**: A generated agent must never go live on a real client's calls without a human approving it first. This needs a review/approval UI and status (`draft` → `pending_review` → `approved` → `live`), not a "generate and done" flow.
- **Versioning**: Every generated/edited prompt needs a version history and the ability to roll back — a bad version here affects live calls with real customers, so this can't just be a "generate and overwrite" flow.
- **Evaluation**: Before launch, a small regression suite of test conversations (adversarial objections, edge cases like disputed debt) should be run against any new agent prompt to catch obviously broken behavior before a human even reviews it.

3.2 Cross-client outcomes dashboard (Task 2)

- **Data integration**: Replace the static JSON with a real query against Domu's call data store (whatever logs call outcomes today — likely a Postgres table or a data warehouse fed from the telephony layer). This is the most straightforward piece to productionize.
- **Freshness & caching**: Decide an acceptable staleness window (e.g. refreshed every 15 minutes vs. real-time) and cache accordingly — recomputing aggregates from millions of call rows on every dashboard load won't scale.
- **Date range filtering**: The MVP's expandable per-client detail (avg handle time, top failure reason, calls by day) is still all-time; production needs real "this week vs. last week" comparisons, not

just a snapshot.
- **CSV export**: Already works client-side for the current page of data; at real volume this should become a server-generated export job instead of exporting whatever happens to be loaded in the browser.

3.3 QA triage of flagged calls (Task 3)

- **Data integration**: Pull from wherever calls actually get flagged today (a QA queue, a low-confidence-outcome heuristic, or manual supervisor flags) instead of a static list.
- **Transcript access**: Real transcripts (and ideally the audio) need to be available to the classifier, with proper access control — this is sensitive customer data in a regulated industry.
- **Correction logging**: The MVP already treats the AI's category as a suggestion, not a verdict — a human picks the final category, and the AI badge stays visible so the choice is visibly informed rather than automatic. Production needs to actually store both values (AI suggestion vs. human's final pick) per call, so agreement rate between the two can be tracked over time — that's the real signal for whether the AI is good enough to lean on more.

3.4 Fix poor objection handling (Task 4)

Same pattern as 3.3: the MVP shows a queue of client-reported issues (current prompt + problem description already attached, not typed by hand), the Technical Ops Lead writes the revised prompt directly — optionally with an AI-assisted diagnosis and draft to start from — and Saving moves it into a "Fixed" table that stays editable afterward.

- **Real issue source**: The queue is a static mock list today; production should source it from wherever objection-handling complaints actually get logged — plausibly Task 3's QA triage output itself (a cluster of "incorrect_statement" flags or a recurring complaint could surface here automatically instead of someone writing it up from scratch).
- **Pulling the current live prompt automatically**: Once 3.1's

versioning system exists, the "current prompt" shown here should be pulled live from whichever client/agent is selected, not from a static snapshot.
- **Same review/versioning gate as 3.1**: A revised prompt from this flow is exactly as sensitive as one from Task 1 — it needs the same approval step and version history before it reaches a live call. Right now "Save" just marks it fixed in this tool; production needs that to actually trigger 3.1's `draft → pending_review → approved → live` flow.
- **Correction/agreement tracking**: Same signal as 3.3 — since both the AI's suggested fix and the human's final version are captured, production could track how often the Ops Lead accepts the AI draft as-is vs. rewrites it, as a read on whether it's actually saving time.

3.5 Client request → engineering ticket (Task 5)

The MVP also lets the Ops Lead add context the client didn't say (e.g. "only affects customers in Texas") — that detail gets woven into the ticket's description and added as its own acceptance criterion, not just tacked onto a notes field where it'd be easy to miss. The draft can be copied, or exported as Markdown, plain text, or PDF, whichever's easiest to paste somewhere else before a real tracker integration exists.

- **Integration**: Push directly into whatever issue tracker Domu's engineering team uses (Linear, Jira, GitHub Issues) via API. The MVP's "Send to tracker" button is a placeholder that simulates this — swapping it for a real API call is a small, contained piece of work once the target tracker and its auth are decided.
- **Client/account linkage**: Auto-attach the relevant client account, current agent version, and any related past tickets for context — a developer picking this up will want that immediately, not have to go look it up.
- **Guardrail**: Already reflected in the MVP's framing ("draft for you to review and edit, nothing gets filed automatically") — production should keep that same rule: the draft is never auto-filed without a human looking at it first.

3.6 Compliance concern investigation (Task 6)

The highest-stakes task in this list, and the MVP treats it that way: the queue of escalated concerns comes with the transcript and the client's concern already attached (not typed by hand), every AI-assisted output is labeled as a draft for a human compliance/legal reviewer, the prompt is instructed to never state a firm legal conclusion, and Saving just moves the Ops Lead's own notes into a "Completed investigations" table — nothing gets sent anywhere from this tool. For production, that posture needs to become procedural, not just a UI pattern:

- **Real concern source**: The queue is a static mock list today; production needs this fed by however clients actually escalate a concern to Domu (support ticket, email, a dedicated escalation form) rather than a fixed list.
- **Mandatory legal/compliance sign-off**: No output from this feature should reach a client, customer, or regulator without an actual human compliance reviewer approving it — this needs to be enforced by workflow (a required approval step tied to a real reviewer identity), not just implied by the UI copy or the fact that "Save" only saves it *in this tool*.
- **Audit trail**: Every investigation, the AI's draft (if used), the reviewer's edits, and the final approved version all need to be logged and retained — this is exactly the kind of record a regulator could ask for later.
- **Escalation path, not a self-serve tool**: Worth an explicit conversation with Domu's legal team before this goes further — they may want this gated behind a formal escalation process rather than available to any Technical Ops Lead.

3.7 Calling windows & holiday check (Task 7)

The most deterministic task here — a rules engine, not an LLM problem — which is exactly how the MVP built it (see `backend/app/routers/calling_compliance.py`). For production:

- **Real, sourced rules**: The permitted-hour windows and holiday list in the MVP are illustrative demo data, explicitly labeled as such in the UI. Production needs these sourced from Domu's actual

compliance team, per region, and kept current — getting this wrong isn't a UX bug, it's a real compliance risk.
- **Real timestamps & timezones**: The MVP treats each attempt's timestamp as already being local time for its region. Production needs to store attempts in UTC and convert properly per region's actual timezone (including daylight saving), not assume the stored value is already local.
- **Proactive check needs to persist and actually gate dialing**: The MVP already added a "check before you call" step — a proposed call gets checked and added straight into the same list as past attempts, not shown as a disconnected one-off result. But it's still client-side only (the check disappears on refresh) and advisory (nothing stops the call from actually being dialed). Production needs this check to persist server-side and to be wired into whatever actually places the call, so a violation blocks the dial rather than just getting logged after the fact.

3.8 Audio → transcript (feeds into 3.1 and 3.4)

The MVP includes a working local speech-to-text step (upload a call recording, get a speaker-labeled transcript) tested against two real Domu call recordings, so a Technical Ops Lead can start from an actual recording instead of typing a script from scratch. For production:

- **Managed transcription service**: the local model here (chosen to avoid needing another paid API key for a proof-of-concept) doesn't belong in production — a managed service (e.g. Deepgram, AssemblyAI, or whatever Domu's telephony layer already produces) would give better accuracy, language coverage, and no per-request model load time.
- **Async processing**: transcribing a multi-minute call inline, in the same request that returns the result, does not scale — this should be a background job (upload → queued job → notify when ready), especially once call recordings are minutes long and processed in bulk.
- **Likely redundant with existing data**: Domu's platform almost certainly already transcribes every call for its own purposes — the real integration point is probably "pull the existing transcript" rather than "re-transcribe audio," and this step becomes unnecessary

once 3.2's data integration exists.
- **Speaker labeling is currently a heuristic**: the MVP splits stereo channels and assumes whichever channel speaks first is the agent (true for every call in this flow, since the agent always opens with the greeting) — this holds for Domu's actual recording setup, but should be validated against real recording pipelines before being relied on.

4. Cross-cutting production requirements

- **Multi-tenancy & access control**: Every client's data (transcripts, outcomes, agent prompts) must be isolated per account, with role-based access — a Technical Ops Lead should only see the clients they're assigned to.
- **Auditability**: Given the regulated nature of the clients (banks, insurers), every AI-generated artifact that could end up affecting a live call or a client-facing decision (agent prompt, compliance response) needs an audit trail: who generated it, who approved it, when it went live.
- **Reliability at the LLM layer**: The current wrapper (timeout + one retry + graceful fallback) is a reasonable MVP baseline but would need, for production: request-level rate limiting, circuit breaking if the provider degrades broadly, and cost monitoring (LLM spend scales with usage in a way a fixed infra bill doesn't).
- **Observability**: Structured logging and basic metrics (LLM latency, failure rate, fallback rate) from day one — when this breaks at 2am during a client's calling window, someone needs to know why without reading application code.
- **Security/compliance**: PII (names, account details, payment info) shows up in call transcripts. Encryption at rest/in transit, and scoped access to raw transcripts, should be in place before this touches real client data — aligned with the SOC2/ISO27001 posture the role description mentions.
- **Testing**: Beyond the unit tests included here, production needs integration tests against a staging LLM environment and, ideally, a small labeled eval set per feature to catch quality regressions when prompts change.

- **Client-side PDF export weight**: Generating PDFs in the browser (Tasks 1 and 5) pulls in a library with heavier dependencies than the MVP actually uses (~240KB gzipped, lazy-loaded only when someone clicks "Download PDF," so it doesn't affect normal page load). Fine for an internal tool; if this needs to be leaner, a server-side PDF render would be the fix.

5. Suggested phased rollout

1. **Phase 1** — Wire Task 2 (dashboard) to real data. Lowest risk, no LLM output reaching a live call, immediate value for weekly reporting.
2. **Phase 2** — Task 7 (calling windows/holiday check) with real, compliance-sourced rules and proper timezone handling. Also low-risk (deterministic, no LLM), and directly reduces regulatory exposure.
3. **Phase 3** — Task 3 (QA triage) with correction logging, to start building the accuracy-tracking signal described in 3.3.
4. **Phase 4** — Task 5 (ticket generation), integrated with the real issue tracker.
5. **Phase 5** — Task 1 (script → agent) with the review/approval gate and versioning, since this is the first feature whose output can end up on a live call.
6. **Phase 6** — Task 4 (fix objection handling), once 3.1's versioning/eval infrastructure and 3.3's QA data exist to build on.
7. **Phase 7** — Task 6 (compliance investigation) last, and only after an explicit conversation with legal/compliance about the required sign-off process.

6. Open questions for engineering/product before starting

- What's the actual system of record for call outcomes and flagged calls today — is there already an API/warehouse to integrate against, or does that need to be built first?

- What issue tracker does engineering use, and is there an existing API integration pattern to follow?
- Who has authority to approve an agent prompt going live for a client — is that the Technical Ops Lead alone, or does it need a second sign-off given the regulated context?
- Who in legal/compliance would own sign-off on Task 6's output, and what does that approval workflow need to look like?
- What are the actual, current permitted-calling-hour rules per region Domu operates in, and who maintains that list as it changes?
- Is there an existing eval/regression framework for voice agent behavior, or would this be the first one?